About

This practice lab is about practicing variables and expressions using Python. These practice labs are counted as extra credits and should give you a good opportunity to hone your programming skills before the labs.

Learning Objectives

1. Learn to use the `input()` and `print()` functions
2. Practice using variables to store information
3. Practice basic arithmetic expressions

Estimated Size

3 small programs/files. While working on each question, you can submit a file individually to Gradescope to verify your work. In the end, you must submit all 3 files together to get full credit.

Credit

Failing to submit this assignment will **not** result in a decrease in your lab grade. These assignments are **optional extra credit assignments.**

Tips - Preamble

The `input()` and `print()` functions will be integral to this assignment.

`input()` prompts the user for input, and returns what the user has typed as a string. The `input()` function can optionally take a string as the prompt text/message. So the command

```
s = input("Enter Input: ")
```

would print out **`Enter Input:`** to the terminal, await the input, and then store the text the user entered as a string in variable `s`.

Since `input()` always returns a string, if you want it converted to a number to use in arithmetic expressions, you can use **type casting** to convert it into an `int` or `float`.

`print()` can take in data or an expression of any type as a parameter, and print it to the terminal. So

```
print("Received: " + s)
```

(assuming `s` is a string) would print `Received:` followed by the content of `s`. It can also take multiple parameters and print them on the same line, separated by a space character in between. For example

```
print(1, "abc", s)
```

will print the number `1`, the string `"abc"`, and the content of variable `s` on the same line, separated by the space character.

`print()` always ends with a new line. So

```
print(1)
print("abc")
```

```
print(s)
```
will print the three items on separate lines. If you want to print a number `n` following some text:
```
print("n: " + str(n))
```
will work (note the **type casting**: the integer `n` must be converted to a string in order to be concatenated with another string).

0. Setup

In your Python files, you must include author information. On the first line, add a **`# Author`** comment line, where you mention your full name. Similarly, include your email and Spire ID on the **`# Email`** and **`# Spire ID`** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

1. A starting example `echo.py` (1 point)

The first program is provided to you as a starting example. It's already implemented. All you need to do is download the file, open it in VSCode, and change the first three lines of comments to include your author information. You do NOT need to change the program code. This example reads in a user input, stores it in a variable, and prints it back to the terminal. After you have edited the comments, save it. You can then run it, try some different inputs, and verify the outputs are correct.

File download: echo.py

2. Implement `ab.py` (4 points)

This is a new program you must create from scratch. In VSCode, create a new file and name it `ab.py`. Note that the file name is **case-sensitive**: all letters must be lower-case. Naming it `Ab.py` or `AB.py` would NOT work. Next, include the three lines of comments at the beginning of this file.

The task of this program is to read in two inputs from the user (using the input function):

1. A string (let's call it "a")
2. A string (let's call it "b")

The program should then output the following pattern:

- The string "a" followed by the string "b"

Running a correct implementation of this program should look like:

```
Enter the string a: Hello
Enter the string b: Hi
HelloHI
```

The prompt text/flavor text of your `input()` function does not have to exactly match the example above. The autograder does not check it, so you can use any reasonable prompt text as you want. **But make sure the output is only as asked. Do not output anything extra.** After finishing the program, run it a few times, try different inputs, and verify the outputs are correct.

3. Implement `arithmetic.py` (5 points)

Create a new program, name it `arithmetic.py`. Next, include the three lines of comments at the beginning of this file. The task of this program is to read in two inputs from the user, store each in a separate variable:

1. A positive integer (let's call it "a")
2. Another positive integer (let's call it "b")

The program should then:

1. Print the results of the following arithmetic operations between "a" and "b".
    - Addition
    - Multiplication
    - Division
    - Integer Division
    - Remainder

You may add other strings to make the flow of your output better. Take a look at the examples.

Running a correct implementation of this program may look like:

```
Enter a: 15
Enter b: 6
Addition: 21
Multiplication: 90
Division: 2.5
Integer Division: 2
Remainder: 3
```

Another example:

```
Enter a: 150
Enter b: 6
Addition: 156
Multiplication: 900
Division: 25.0
Integer Division: 25
```

```
Remainder: 0
```

Output requirement: There should be exactly 5 lines of output. Every line of your output must contain one and only one number, and it must be the result of the corresponding arithmetic operation for that line. Other text/formatting does not matter. The autograder only checks the correctness of the numbers and ignores other texts.

4. Submit the three Python(`.py`) files to Gradescope

Log in to Gradescope, select **Practice for Lab 01: Variables and Expressions (code)**, and submit **all of** `echo.py`, `ab.py`, and `arithmetic.py` there **at once**. You can do so in several ways:

- Select all three files, and drag and drop them into the submission area when prompted.
- Or, browse your files when prompted by Gradescope and select all files you want to submit.
- Alternatively, compress all the files you want to submit into a single zip file (the zip file name does not matter); then submit that zip file via drag-and-drop or browsing.
- You can also compress the folder that contains all the files you want to submit as a single zip file (the zip file name does not matter), and upload that zip file via drag-and-drop or browsing.

If you don't submit all required files at once, you won't get credit for the missing files, but you do still get partial credit for the ones you submitted.
If you run into any trouble submitting, Gradescope has a guide on submitting assignments like this.
Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.